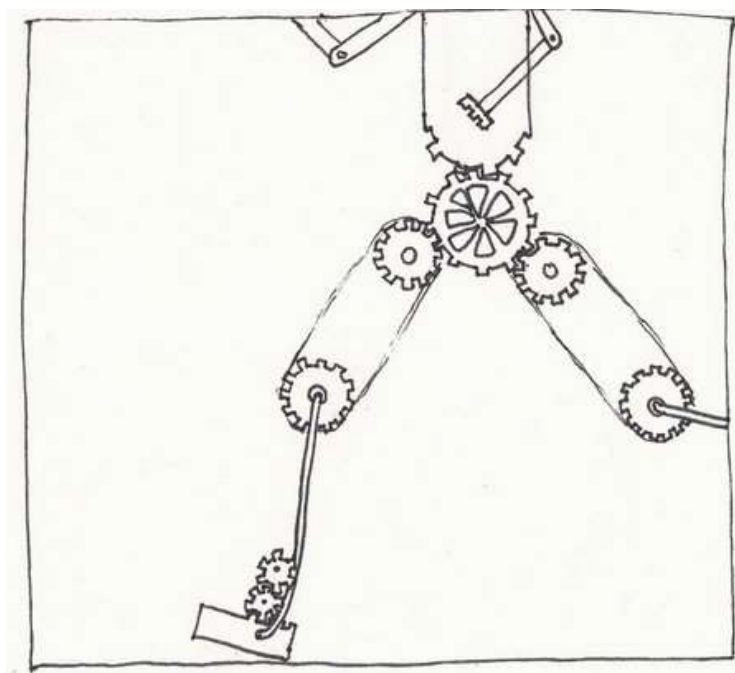


תרגיל 1 – חישוב זמן ריצה של תכניות מחשב



זמן ריצה היא פונקציה.

הקלט לפונקצית זמן הריצה: גודל הקלט (כמות הנתונים הבסיסיים) לקטע הקוד.

הפלט של פונקציית זמן הריצה: כמות הפעולות הבסיסיות שמבצע קטע הקוד.

פעולה בסיסית

היחידה האריתמטית-לוגית מסוגלת לבצע שלושה סוגים של פעולות בסיסיות (הרחבה תלמד במסגרת הקורס מבנה המחשב ותורת המיתוג):

- פעולות אריתמטיות

למשל, חיבור או חיסור שני מספרים. אוסף הפעולות האריתמטיות שיכולה היחידה לבצע עלול להיות דל ביותר עד כדי מצב שבו בחלק מן הדגמים לא מומשה ישירות האפשרות של כפל וחילוק (במקום מימוש ישיר בחומרה, יש להשתמש בתוכנות המבצעות כפל וחילוק באמצעות אלגוריתם).

- פעולות על מחרוזות בזיכרון המחשב

- פעולות לוגיות

פעולות השוואה בעיקר. למשל, בהינתן שני מספרים, קבע האם הם שווים, ואם לא, מי מהם גדול יותר.

חשבו את פונקציית זמן הריצה המדויקת של התכניות הבאות כקלט של n
(נניח לצורך התרגיל כי `Printf()` היא פעולה בסיסית):

```
void fun(int *A, int n)
{
    int i;
    for (i = 0; i < n; i++)
        printf("%d,", A[i]);
}
```

```
void fun(int *A, int n)
{
    int i;
    0..(n-1) for (i = 0; i < n; i++)
               printf("%d,", A[i]);
}
```

```
void fun(int *A, int n)
{
    int i;
    for (i = 0; i < n; i++)
        printf("%d,", A[i]);
}
```

n +1

```
void fun(int *A, int n)
{
    int i;
    for (i = 0; i < n; i++)
        printf("%d,", A[i]);
}
```

n +1

n


```
void fun(int *A, int n)
{
```

```
    int i;
```

n +1

```
    for (i = 0; i < n; i++)
```

n

```
        printf("%d,", A[i]);
```

$T(n)=2 \cdot n+1$ }

```
void fun(int *A, int n)
{
    int i;
    for (i = 0; i < n; i+=2)
        printf("%d,", A[i]);
}
```

האם קיים הבדל בין כמות קלט זוגית לכמות אי זוגית ?

```
void fun(int *A, int n)
{
    int i;
    for (i = 0; i < n; i+=2)
        printf("%d,", A[i]);
}
```

נבדוק עבור $n=10$

```
void fun(int *A, int n)
{
    int i;
    0,2,4,6,8,10 for (i = 0; i < n; i+=2)
                  printf("%d,", A[i]);
}
```

נבדוק עבור $n=10$

5

הצלחות

0,2,4,6,8

```
void fun(int *A, int n)
{
    int i;
    for (i = 0; i < n; i += 2)
        printf("%d,", A[i]);
}
```

נבדוק עבור $n=11$

```
void fun(int *A, int n)
{
    int i;
    0,2,4,6,8,10 for (i = 0; i < n; i+=2)
                  printf("%d,", A[i]);
}
```

נבדוק עבור $n=11$

6

הצלחות

0,2,4,6,8,10

```
void fun(int *A, int n)
{
    int i;
    for (i = 0; i < n; i += 2)
        printf("%d,", A[i]);
}
```

מהי פונקציית זמן הריצה ?

```
void fun(int *A, int n)
{
    int i, j, k = 0;
    for (i = 0; i < n; i++)
        for (j = 0; j < n/2; j++) {
            k += 1;
            A[i] = k
        }
}
```




בניח ש-n
זוגי

0,1,2,3,...,n-1

0,1,2,3,...,n/2-1

```
void fun(int *A, int n)
{
    int i, j, k = 0;
    for (i = 0; i < n; i++)
        for (j = 0; j < n/2; j++) {
            k += 1;
            A[i] = k
        }
}
```

בניח ש- n
זוגי

$0, 1, 2, 3, \dots, n-1$

$0, 1, 2, 3, \dots, n/2-1$

```
void fun(int *A, int n)
{
    int i, j, k = 0;
    for (i = 0; i < n; i++)
        for (j = 0; j < n/2; j++) {
            k += 1;
            A[i] = k
        }
}
```

מה מספר ההצלחות ומה מספר הכישלונות?

בניח ש- n
זוגי

$0, 1, 2, 3, \dots, n-1$

$0, 1, 2, 3, \dots, n/2-1$

```
void fun(int *A, int n)
{
    int i, j, k = 0;
    for (i = 0; i < n; i++)
        for (j = 0; j < n/2; j++) {
            k += 1;
            A[i] = k
        }
}
```

מהי פונקציית זמן הריצה ?

```
void fun(int *A, int n)
{
    int i, j, k = 0;
    for (i = 1; i < n; i*=2)
        for (j = 1; j < n/2; j*=4) {
            k += 1;
            A[i] = k
        }
}
```

```
void fun(int *A, int n)
{
    int i, j, k = 0;
    for (i = 1; i < n; i*=2)
        for (j = 1; j < n/2; j*=4) {
            k += 1;
            A[i] = k
        }
}
```

1

```
void fun(int *A, int n)
{
    int i, j, k = 0;
    for (i = 1; i < n; i*=2)
        for (j = 1; j < n/2; j*=4) {
            k += 1;
            A[i] = k
        }
}
```

1

הנח ש- n
חזקה של 2



1

```
void fun(int *A, int n)
{
    int i, j, k = 0;
    for (i = 1; i < n; i*=2)
        for (j = 1; j < n/2; j*=4) {
            k += 1;
            A[i] = k
        }
}
```

הנח ש- n
חזקה של 2


$$n = 2^{\log n}$$

1

```
void fun(int *A, int n)
{
    int i, j, k = 0;
    for (i = 1; i < n; i*=2)
        for (j = 1; j < n/2; j*=4) {
            k += 1;
            A[i] = k
        }
}
```


הנח ש- n
חזקה של 2

$$n = 2^{\log n}$$

1

1, 2, 4, 8, ..., $2^{\log n - 1}$

```
void fun(int *A, int n)
{
    int i, j, k = 0;
    for (i = 1; i < n; i *= 2)
        for (j = 1; j < n/2; j *= 4) {
            k += 1;
            A[i] = k
        }
}
```

הנח ש- n
חזקה של 2

$$n = 2^{\log n}$$

1

1, 2, 4, 8, ..., $2^{\log n - 1}$

```
void fun(int *A, int n)
{
    int i, j, k = 0;
    for (i = 1; i < n; i*=2)
        for (j = 1; j < n/2; j*=4) {
            k += 1;
            A[i] = k
        }
}
```

הנח ש- n
חזקה של 2

$$n = 2^{\log n}$$

1

 $1, 2, 4, 8, \dots, 2^{\log n - 1}$
 $2^0, 2^1, 2^2, 2^3, \dots, 2^{\log n - 1}$

```
void fun(int *A, int n)
```

```
{
```

```
    int i, j, k = 0;
```

```
    for (i = 1; i < n; i*=2)
```

```
        for (j = 1; j < n/2; j*=4) {
```

```
            k += 1;
```

```
            A[i] = k
```

```
        }
```

```
    }
```

הנח ש- n
חזקה של 2

$$n = 2^{\log n}$$

1

1, 2, 4, 8, ..., $2^{\log n - 1}$

$2^0, 2^1, 2^2, 2^3, \dots, 2^{\log n - 1}$

```
void fun(int *A, int n)
```

```
{
```

```
    int i, j, k = 0;
```

```
    for (i = 1; i < n; i*=2)
```

```
        for (j = 1; j < n/2; j*=4) {
```

```
            k += 1;
```

```
            A[i] = k
```

```
        }
```

```
    }
```

הנח ש- n
חזקה של 2

$$n = 2^{\log n}$$

1

 $2^0, 2^1, 2^2, 2^3, \dots, 2^{\log n - 1}$

```
void fun(int *A, int n)
```

```
{
```

```
    int i, j, k = 0;
```

```
    for (i = 1; i < n; i*=2)
```

```
        for (j = 1; j < n/2; j*=4) {
```

```
            k += 1;
```

```
            A[i] = k
```

```
        }
```

```
    }
```

הנח ש- n
חזקה של 2

$$n = 2^{\log n}$$

1

 $\log n$

```
void fun(int *A, int n)
{
    int i, j, k = 0;
    for (i = 1; i < n; i*=2)
        for (j = 1; j < n/2; j*=4) {
            k += 1;
            A[i] = k
        }
}
```

הנח ש- n

חזקה של 4



$$n = 4^{\log n}$$

1

 $\log n$

```

void fun(int *A, int n)
{
    int i, j, k = 0;
    for (i = 1; i < n; i*=2)
        for (j = 1; j < n/2; j*=4) {
            k += 1;
            A[i] = k
        }
}

```

הנח ש- n

חזקה של 4

$$n = 4^{\log_4 n}$$

1

 $\log n$

```

void fun(int *A, int n)
{
    int i, j, k = 0;
    for (i = 1; i < n; i*=2)
        for (j = 1; j < n/2; j*=4) {
            k += 1;
            A[i] = k
        }
}

```


הנח ש- n

חזקה של 4



$$n = 4^{\log_4 n} = 2^{2 \cdot \log_4 n}$$

1

 $\log n$

```
void fun(int *A, int n)
```

```
    int i, j, k = 0;
```

```
    for (i = 1; i < n; i*=2)
```

```
        for (j = 1; j < n/2; j*=4) {
```

```
            k += 1;
```

```
            A[i] = k
```

```
        }
```

```
    }
```

הנח ש-n

חזקה של 4

$$n = 4^{\log_4 n} = 2^{2 \cdot \log_4 n}$$

1

 $\log n$
 $4^0, 4^1, 4^2, 4^3, \dots, 4^{\log_4 n - 0.5}$

```
void fun(int *A, int n)
```

```
    int i, j, k = 0;
```

```
    for (i = 1; i < n; i*=2)
```

```
        for (j = 1; j < n/2; j*=4) {
```

```
            k += 1;
```

```
            A[i] = k
```

```
        }
```

```
    }
```

הנח ש-n

חזקה של 4



$$n = 4^{\log_4 n} = 2^{2 \cdot \log_4 n}$$

1

 $\log n$
 $4^0, 4^1, 4^2, 4^3, \dots, 4^{\log_4 n - 0.5}$

```
void fun(int *A, int n)
```

```
    int i, j, k = 0;
```

```
    for (i = 1; i < n; i*=2)
```

```
        for (j = 1; j < n/2; j*=4) {
```

```
            k += 1;
```

```
            A[i] = k
```

```
        }
```

```
    }
```

מה מספר ההצלחות ומה מספר הכישלונות?

הנח ש-n

חזקה של 4

$$n = 4^{\log_4 n} = 2^{2 \cdot \log_4 n}$$

1

 $\log n$
 $4^0, 4^1, 4^2, 4^3, \dots, 4^{\log_4 n - 0.5}$

```
void fun(int *A, int n)
```

```
    int i, j, k = 0;
```

```
    for (i = 1; i < n; i*=2)
```

```
        for (j = 1; j < n/2; j*=4) {
```

```
            k += 1;
```

```
            A[i] = k
```

```
        }
```

```
    }
```

מהי פונקציית זמן הריצה ?

```
void fun(int *A, int n)
{
    int i, j, k0 = 0, k1 = 1;
    for (i = 0; i < n; i++)
        for (j = 0; j < i; j++)
        {
            k0 += 1;
            k1 *= 2;
            A[i] = k0 + k1;
        }
}
```

```
void fun(int *A, int n)
{
    int i = 0;
    while (i < n) {
        A[i] += 1;
        i++;
    }
}
```

```
void fun(int *A, int n)
{
    int i = 0;
    while (i < n) {
        A[i] += 1;
        i += 3;
    }
}
```

```
void fun(int *A, int n)
{
    int i = 1;
    while (i < n) {
        A[i]++;
        i *= 2;
    }
}
```

תרגיל 8


```
void fun(int *A, int n)
{
    int i = 2;
    while (i < n) {
        A[i] = A[i - 1]; // כל השורה הנ"ל מהווה פעולה בסיסית אחת
        i = i * i;
    }
}
```

```
void fun(int *A, int n)
{
    int k;
    k = n;
    while (k > 1) {
        k = k/5;
        for (j = 1; j <= log(n); j++)
            for (i = 1; i <= n/2; i++)
                A[i] += 1;
    }
}
```

```
void fun(int *A, int n)
{
    int k = 2, m = 0;
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= i; j++)
            m += 1;
        while (k <= i)
            k = k * k;
    }
    A[m%n] += k; // כל השורה הנ"ל מהווה פעולה בסיסית אחת
}
```